ARTIFICIAL INTELLIGENCE LAB

Name: P .Lakshmi Gayathri Navaneetha
Reg No:192321169

EXP. 1 — 8-PUZZLE PROBLEM (CO1)

Aim

To solve the 8-puzzle problem using Breadth-First Search (BFS). The blank tile is represented by 0.

Algorithm

1. Read the initial 3×3 puzzle and define the goal state.
2. Put the initial state in a queue and mark it as visited.
3. Remove the first state from the queue.
4. If it matches the goal state, stop and display the path.
5. Generate valid states by moving the blank tile up, down, left, or right.
6. Add each unvisited state to the queue and repeat until the goal is found or the queue is empty.

Python Code

```
from collections import deque

start = tuple(map(int, input("Enter 9 numbers (use 0 for blank): ").split()))
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def neighbors(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc, move in [(-1,0,"Up"), (1,0,"Down"),
                         (0,-1,"Left"), (0,1,"Right")]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            nz = nr * 3 + nc
            s = list(state)
            s[z], s[nz] = s[nz], s[z]
            yield tuple(s), move

queue = deque([(start, [])])
visited = {start}
solution = None

while queue:
```

```
    state, path = queue.popleft()
    if state == goal:
        solution = path
        break
    for nxt, move in neighbors(state):
        if nxt not in visited:
            visited.add(nxt)
            queue.append((nxt, path + [(nxt, move)]))

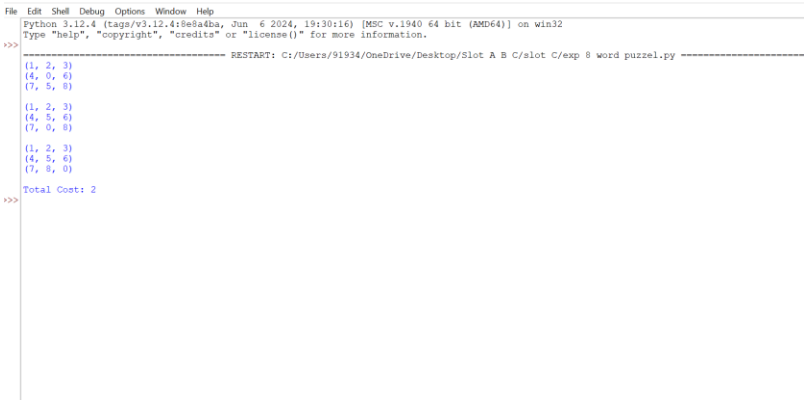
if solution is None:
    print("No solution found (or puzzle is unsolvable).")
else:
    print("Moves:", len(solution))
    for state, move in solution:
        print(move)
        for i in range(0, 9, 3):


            print(state[i:i+3])
```

Sample Input

1 2 3 4 5 6 7 0 8

Sample Output



EXP. 2 — 8-QUEEN PROBLEM (CO1)

Aim

To place eight queens on an 8×8 chessboard so that no two queens attack each other, using backtracking.

Algorithm

7. Start with an empty chessboard.
8. Place one queen in the current row.
9. Check that its column and both diagonals are not occupied by earlier queens.
10. If the position is safe, move to the next row.
11. If no position is safe, backtrack to the previous row and try another column.
12. When all eight rows are filled, display the solution.

Python Code

```
N = 8
board = [-1] * N

def safe(row, col):
    for r in range(row):
        if board[r] == col or abs(board[r] - col) == abs(r - row):
            return False
    return True

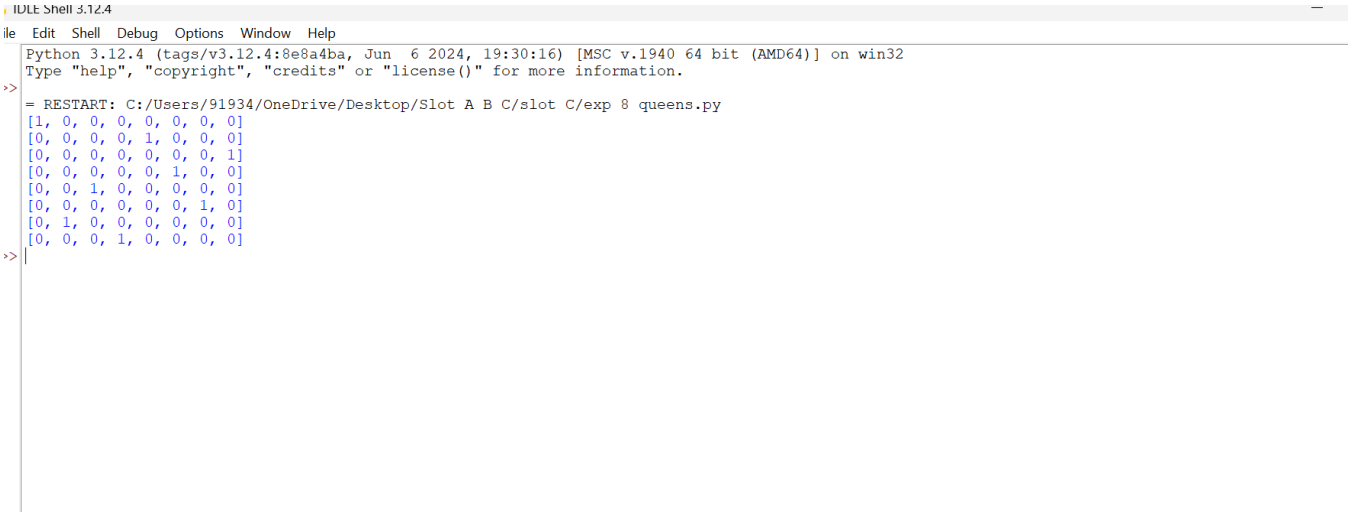
def solve(row):
    if row == N:
        return True
    for col in range(N):
        if safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False

if solve(0):
    for r in range(N):
        print(" ".join("Q" if board[r] == c else "." for c in range(N)))
else:
    print("No solution found.")
```

Sample Input

No input required.

Sample Output



EXP. 3 — WATER JUG PROBLEM (CO2)

Aim

To measure a target quantity of water using two jugs and BFS state-space search.

Algorithm

13. Read the capacities of both jugs and the target quantity.
14. Represent each state as the amount of water in jug A and jug B.
15. Start BFS from (0, 0) and record each visited state.
16. Generate states by filling, emptying, or pouring between jugs.
17. Stop when either jug contains the target amount.
18. Display the sequence of states, or report that the target cannot be measured.

Python Code

```
from collections import deque

A = int(input("Capacity of jug A: "))
B = int(input("Capacity of jug B: "))
target = int(input("Target amount: "))

start = (0, 0)
queue = deque([start])
parent = {start: None}
goal = None

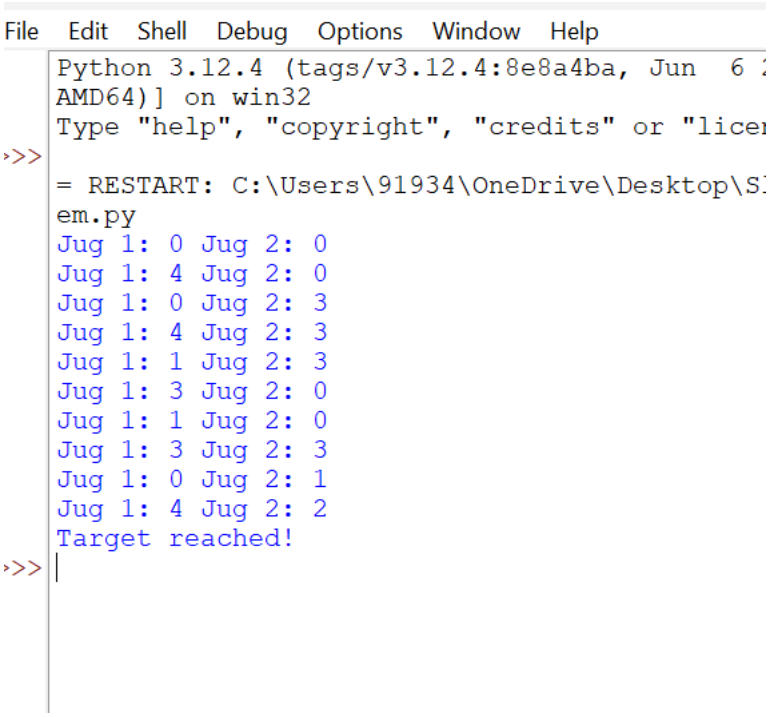
while queue:
    x, y = queue.popleft()
    if x == target or y == target:
        goal = (x, y)
        break
    pour_ab = min(x, B - y)
    pour_ba = min(y, A - x)
    next_states = [
        (A, y), (x, B), (0, y), (x, 0),
        (x - pour_ab, y + pour_ab),
        (x + pour_ba, y - pour_ba)
    ]
    for state in next_states:
        if state not in parent:
            parent[state] = (x, y)
            queue.append(state)

if goal is None:
    print("Target cannot be measured.")
else:
    path = []
```

```
    cur = goal
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    print("Steps (A, B):")
    for state in reversed(path):
        print(state)
```

Sample Input
Capacity of jug A: 4
Capacity of jug B: 3
Target amount: 2

Sample Output



EXP. 4 — CRYPTARITHMETIC PROBLEM (CO2)

Aim
To solve the cryptarithmetic puzzle SEND + MORE = MONEY by assigning a unique digit to each letter.

Algorithm
19. List the distinct letters in the puzzle.
20. Try permutations of digits 0–9, assigning one digit to each letter.
21. Reject assignments where S or M is zero.
22. Convert SEND, MORE, and MONEY into numbers.

23. Check whether SEND + MORE equals MONEY.
24. Print the first valid digit assignment and its arithmetic result.

Python Code

```
from itertools import permutations

letters = "SENDMORY"
for digits in permutations(range(10), len(letters)):
    d = dict(zip(letters, digits))
    if d["S"] == 0 or d["M"] == 0:
        continue
    SEND = 1000*d["S"] + 100*d["E"] + 10*d["N"] + d["D"]
    MORE = 1000*d["M"] + 100*d["O"] + 10*d["R"] + d["E"]
    MONEY = (10000*d["M"] + 1000*d["O"] + 100*d["N"]
             + 10*d["E"] + d["Y"])
    if SEND + MORE == MONEY:
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        print("Letter mapping:", d)
        break
```

Sample Input

No input required.

Sample Output

```
>>
= RESTART: C:/Users/91934/OneDrive/Desktop/Slot A B C/slot C/cryptarthemic probl
em.py
SEND = 9567
MORE = 1085
MONEY = 10652
Letter mapping: {'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
>>
```

EXP. 5 — MISSIONARIES AND CANNIBALS (CO3)

Aim

To find a safe sequence for transporting three missionaries and three cannibals across a river using BFS.

Algorithm

25. Represent a state as (missionaries_left, cannibals_left, boat_side).
26. Start with (3, 3, 0), where 0 means the boat is on the left.
27. Generate all possible boat loads of one or two people.
28. Reject states outside the banks' limits or where cannibals outnumber missionaries on a bank with missionaries present.
29. Use BFS and store each state's parent to reconstruct the path.
30. Display the safe sequence from the starting state to (0, 0, 1).

Python Code

```
from collections import deque

start = (3, 3, 0)  # 0 = left bank, 1 = right bank
goal = (0, 0, 1)
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

def valid(m, c):
    if not (0 <= m <= 3 and 0 <= c <= 3):
        return False
    mr, cr = 3 - m, 3 - c
    if m > 0 and c > m:
        return False
    if mr > 0 and cr > mr:
        return False
    return True

queue = deque([start])
parent = {start: None}

while queue:
    state = queue.popleft()
    if state == goal:
        break
    m, c, boat = state
    for dm, dc in moves:
        if boat == 0:
            nxt = (m-dm, c-dc, 1)
        else:
            nxt = (m+dm, c+dc, 0)
        if valid(nxt[0], nxt[1]) and nxt not in parent:
```

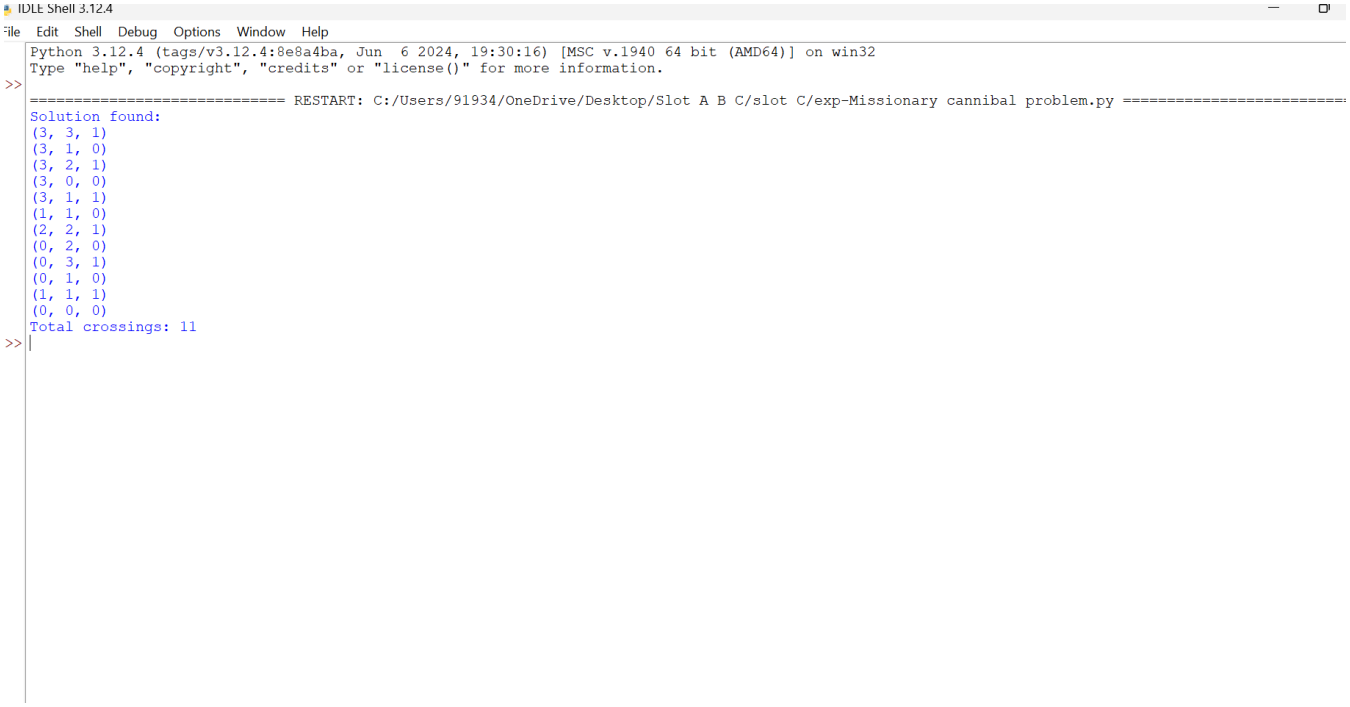
```
            parent[nxt] = state
            queue.append(nxt)

if goal not in parent:
    print("No solution found.")
else:
    path, cur = [], goal
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    for state in reversed(path):
        side = "Left" if state[2] == 0 else "Right"
        print(f"Left bank M={state[0]}, C={state[1]}, Boat={side}")
```

Sample Input

No input required (uses 3 missionaries and 3 cannibals).

Sample Output



EXP. 6 — VACUUM CLEANER PROBLEM (CO3)

Aim

To simulate a simple vacuum-cleaner agent that cleans dirty rooms in a two-room environment.

Algorithm

31. Read the status of rooms A and B as Dirty or Clean.
32. Start the agent in room A.
33. If the current room is dirty, clean it.

34. Move to the other room and clean it if needed.
35. Continue until both rooms are clean.
36. Display the final room status.

Python Code

```
room_a = input("Status of Room A (Dirty/Clean): ").strip().title()
room_b = input("Status of Room B (Dirty/Clean): ").strip().title()

rooms = {"A": room_a, "B": room_b}
for room in ["A", "B"]:
    print("Vacuum is in Room", room)
    if rooms[room] == "Dirty":
        print("Cleaning Room", room)
        rooms[room] = "Clean"
    else:
        print("Room", room, "is already clean")

print("Final status:", rooms)
```

Sample Input

Status of Room A (Dirty/Clean): Dirty
Status of Room B (Dirty/Clean): Clean

Sample Output

```
State 3
Room A: Clean
Room B: Dirty
Vacuum: A
Total Cost: 1

State 4
Room A: Clean
Room B: Dirty
Vacuum: B
Total Cost: 1

State 5
Room A: Dirty
Room B: Clean
Vacuum: A
Total Cost: 1

State 6
Room A: Dirty
Room B: Clean
Vacuum: B
Total Cost: 1

State 7
Room A: Dirty
Room B: Dirty
Vacuum: A
Total Cost: 2

State 8
Room A: Dirty
Room B: Dirty
Vacuum: B
Total Cost: 2

Total States: 8
Goal: Both rooms Clean
>>>
```

EXP. 7 — BREADTH-FIRST SEARCH (BFS) (CO3)

Aim

To implement Breadth-First Search to traverse a graph level by level.

Algorithm

37. Create a graph using an adjacency list.
38. Read or define the starting vertex.
39. Place the start vertex in a queue and mark it visited.
40. Remove a vertex from the front of the queue and display it.
41. Add each unvisited adjacent vertex to the queue and mark it visited.
42. Repeat until the queue is empty.

Python Code

```
from collections import deque

graph = {
   "A": ["B", "C"],
   "B": ["D", "E"],
   "C": ["F"],
   "D": [], "E": [], "F": []
}
start = input("Enter start node (A-F): ").strip().upper()

visited = {start}
queue = deque([start])
order = []

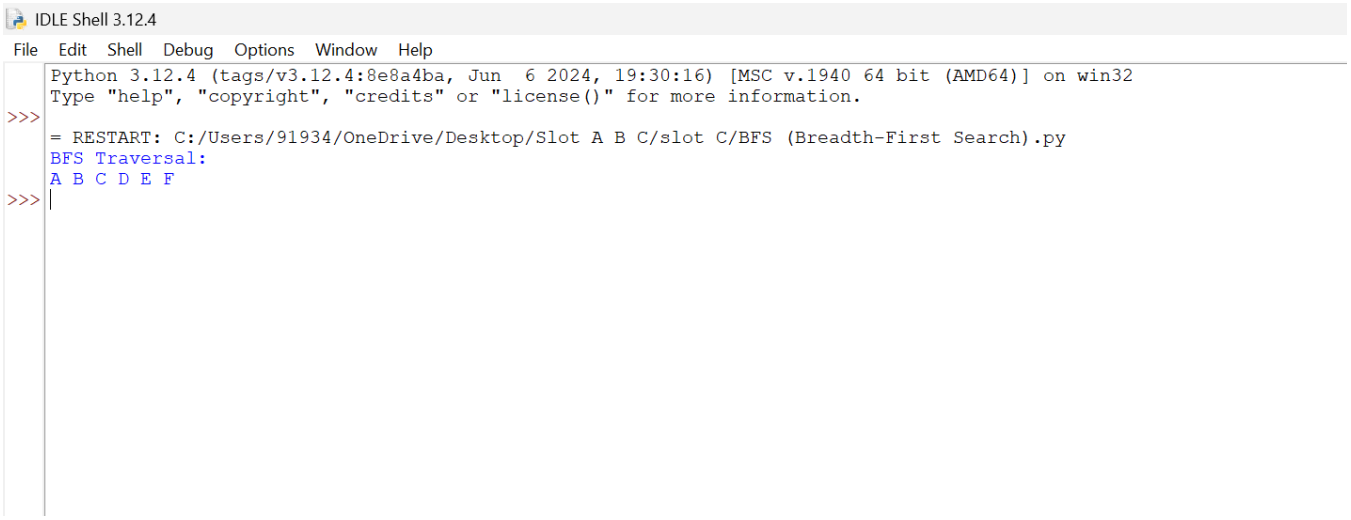
while queue:
   node = queue.popleft()
   order.append(node)
   for neighbor in graph.get(node, []):
      if neighbor not in visited:
         visited.add(neighbor)
         queue.append(neighbor)

print("BFS traversal:", " ".join(order))
```

Sample Input

```
Enter start node (A-F): A
```

Sample Output



EXP. 8 — DEPTH-FIRST SEARCH (DFS) (CO4)

Aim
To implement Depth-First Search to traverse a graph by exploring one branch deeply before backtracking.

Algorithm
43. Create a graph using an adjacency list.
44. Read the starting vertex.
45. Mark the current vertex as visited and display it.
46. Visit an adjacent vertex that has not yet been visited.
47. Continue recursively until no unvisited neighbor remains.
48. Backtrack and continue until all reachable vertices are visited.

Python Code
```
graph = {
   "A": ["B", "C"],
   "B": ["D", "E"],
   "C": ["F"],
   "D": [], "E": [], "F": []
}
start = input("Enter start node (A-F): ").strip().upper()
visited = set()
order = []

def dfs(node):
   if node in visited:
      return
   visited.add(node)
   order.append(node)
   for neighbor in graph.get(node, []):
      dfs(neighbor)

dfs(start)
print("DFS traversal:", " ".join(order))
```

Sample Input
Enter start node (A-F): A

Sample Output

```
>>>
========== RESTART: C:/Users/91934/OneDrive/Desktop/Slot A B C/slot C/Depth frist search.py ==========
DFS Traversal:
A B D E C F
>>>
```

EXP. 9 — TRAVELLING SALESMAN PROBLEM (CO4)

Aim

To find the minimum-cost tour that visits every city exactly once and returns to the starting city, using brute force.

Algorithm

49. Define the cities and the distance matrix.
50. Fix the first city as the starting city.
51. Generate all possible orders of the remaining cities.
52. Calculate each tour's total distance, including the return to the start.
53. Keep track of the tour with the smallest distance.
54. Display the best route and its total cost.

Python Code

```
from itertools import permutations

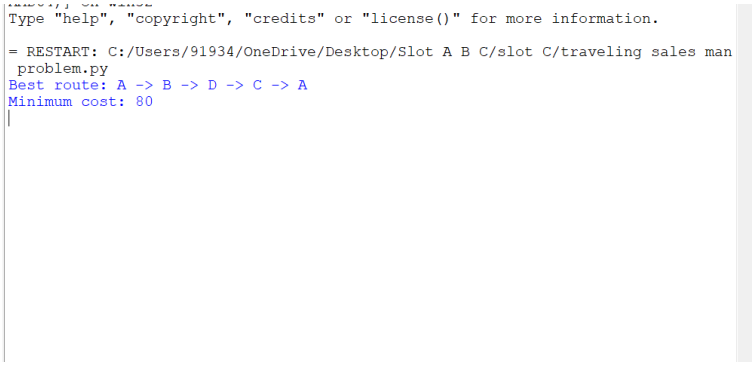
cities = ["A", "B", "C", "D"]
dist = {
   "A": {"A":0, "B":10, "C":15, "D":20},
   "B": {"A":10, "B":0, "C":35, "D":25},
   "C": {"A":15, "B":35, "C":0, "D":30},
   "D": {"A":20, "B":25, "C":30, "D":0}
}

start = cities[0]
best_cost = float("inf")
best_route = None

for order in permutations([c for c in cities if c != start]):
   route = [start] + list(order) + [start]
   cost = sum(dist[route[i]][route[i+1]]
            for i in range(len(route)-1))
   if cost < best_cost:
      best_cost = cost
      best_route = route

print("Best route:", " -> ".join(best_route))
print("Minimum cost:", best_cost)
```

Sample Output



EXP. 10 — A* SEARCH ALGORITHM (CO4)

Aim

To implement the A* search algorithm, which uses path cost g(n) and heuristic estimate h(n) to find a low-cost path.

Algorithm

55. Define the graph with edge costs and a heuristic value for each node.
56. Start with the initial node, with path cost g = 0.
57. Choose the open node with the smallest f(n) = g(n) + h(n).
58. If the selected node is the goal, reconstruct and display the path.
59. Otherwise, update the costs and parents of its neighbors when a cheaper path is found.
60. Repeat until the goal is found or no nodes remain.

Python Code

```
import heapq

graph = {
    "A": {"B": 1, "C": 4},
    "B": {"D": 2, "E": 5},
    "C": {"E": 1},
    "D": {"G": 5},
    "E": {"G": 3},
    "G": {}
}
h = {"A": 7, "B": 6, "C": 4, "D": 4, "E": 2, "G": 0}

def astar(start, goal):
    heap = [(h[start], 0, start)]
    parent = {start: None}
    best_g = {start: 0}
```

```
    while heap:
        f, g, node = heapq.heappop(heap)
        if g != best_g.get(node):
            continue
        if node == goal:
            path, cur = [], goal
            while cur is not None:
                path.append(cur)
                cur = parent[cur]
            return path[::-1], g

        for neighbor, cost in graph[node].items():
            new_g = g + cost
            if new_g < best_g.get(neighbor, float("inf")):
                best_g[neighbor] = new_g
                parent[neighbor] = node
                heapq.heappush(heap, (new_g + h[neighbor],
                                      new_g, neighbor))
    return None, float("inf")

start = input("Start node: ").strip().upper()
goal = input("Goal node: ").strip().upper()
path, cost = astar(start, goal)
if path:
    print("Path:", " -> ".join(path))
    print("Total cost:", cost)
else:
    print("No path found.")
```

Sample Input

Start node: A

Goal node: G

Sample Output

```
>>>
========================================= RESTART: C:/Users/91934/OneDrive/Desktop/Slot A B C/slot C/A=starsearch ==========================================
A* Search:
Shortest Path: ['A', 'B', 'D', 'G']
Total Cost: 6
>>>
```